Manipulate Records with DML

Learning Objectives

After completing this unit, you'll be able to:

- Use DML to insert, update, and delete records.
- Perform DML statements in bulk.
- Use upsert to either insert or update a record.
- Catch a DML Exception.
- Use a Database method to insert new records with the partial success option and process the results.
- Know when to use DML statements and when to use Database methods.
- Perform DML operations on related records.

Manipulate Records with DML

Create and modify records in Salesforce by using the Data Manipulation Language, abbreviated as DML. DML provides a straightforward way to manage records by providing simple statements to insert, update, merge, delete, and restore records.

Because Apex is a data-focused language and is saved on the Salesforce Platform, it has direct access to your data in Salesforce. Unlike other programming languages that require additional setup to connect to data sources, with Apex DML, managing records is made easy! By calling DML statements, you can quickly perform operations on your Salesforce records. This example adds the Acme account to Salesforce. An account sObject is created first and then passed as an argument to the insert statement, which persists the record in Salesforce.

```
1  // Create the account sObject
2  Account acct = new Account(Name='Acme', Phone='(415)555-1212',
3  NumberOfEmployees=100);
4
```

DML Statements

The following DML statements are available.

- `insert` copy
- `update` copy
- `upsert` copy
- `delete` copy
- `undelete` copy
- `merge` copy (available only for the Lead, Contact, Case, and Account sObjects)

Each DML statement accepts either a single sObject or a list (or array) of sObjects. Operating on a list of sObjects is a more efficient way for processing records.

The `insert` copy, `update` copy, and `delete` copy statements are familiar database operations. The `upsert` copy, `undelete` copy, and `merge` copy statements are particular to Salesforce and can be quite handy.

The `upsert` copy DML statement creates new records and updates sObject records within a single statement, using a specified field to determine the presence of existing objects, or the ID field if no field is specified.

The `undelete` copy DML statement restores deleted records that are still in the Recycle Bin (https://help.salesforce.com/s/articleView?id=xcloud.recycle_bin.htm&type=5).

The `merge` copy DML statement merges up to three records of the same sObject type into one of the records, deleting the others, and re-parenting any related records. Only leads, contacts, cases, and accounts can be merged.

ID Field Auto-Assigned to New Records

When inserting records, the system assigns an ID for each record. In addition to persisting the ID value in the database, the ID value is also auto-populated on the sObject variable that you used as an argument in the DML call.

This example shows how to get the ID on the sObject that corresponds to the inserted account.

```
// Create the account sObject
Account acct = new Account(Name='Acme', Phone='(415)555-1212',
NumberOfEmployees=100);
// Insert the account by using DML
```

```
// Debug log result (the ID will be different in your case)
// DEBUG|ID = 001D000000JmKkeIA
```

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Beyond the Basics

Manipulate Records with DML

Because the sObject variable in the example contains the ID after the DML call, you can reuse this sObject variable to perform further DML operations, such as updates, as the system will be able to map the sObject variable to its corresponding record by matching the ID.

You can retrieve a record from the database to obtain its fields, including the ID field, but this can't be done with DML. You'll need to write a query by using SOQL. You'll learn about SOQL in another unit.

Bulk DML

You can perform DML operations either on a single sObject, or in bulk on a list of sObjects. Performing bulk DML operations is the recommended way because it helps avoid hitting governor limits, such as the DML limit of 150 statements per Apex Transaction (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_transaction.htm). This limit is in place to ensure fair access to shared resources on the Salesforce Platform. Performing a DML operation on a list of sObjects counts as one DML statement, not as one statement for each sObject.

This example inserts contacts in bulk by inserting a list of contacts in one call. The sample then updates those contacts in bulk too.

1. Execute this snippet in the Developer Console using Anonymous Apex.

```
// Create a list of contacts
List<Contact> conList = new List<Contact> {
    new
Contact(FirstName='Joe',LastName='Smith',Department='Finance
'),
        new
Contact(FirstName='Kathy',LastName='Smith',Department='Techn
ology'),
        new
Contact(FirstName='Caroline',LastName='Roth',Department='Fin
ance'),
        new
```

```
List<Contact> listToUpdate = new List<Contact>();
// Iterate through the list and add a title only
// if the department is Finance
for (Contact con : conList) {
    if (con.Department == 'Finance') {
```

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Manipulate Records with DML

2. Inspect the contacts recently created in your org.
   Two of the contacts who are in the Finance department now have their titles populated with “Financial analyst.”

Upsert Records

If you have a list containing a mix of new and existing records, you can process insertions and updates to all records in the list by using the `upsert` copy statement. Upsert helps avoid the creation of duplicate records and can save you time as you don’t have to determine which records exist first.

The `upsert` copy statement matches the sObjects with existing records by comparing values of one field. If you don’t specify a field when calling this statement, the `upsert` copy statement uses the sObject’s ID to match the sObject with existing records in Salesforce. Alternatively, you can specify a field to use for matching. For custom objects, specify a custom field marked as external ID. For standard objects, you can specify any field that has the `idLookup` copy property set to true. For example, the Email field of Contact or User has the `idLookup` copy property set. To check a field’s property, see the Object Reference for the Salesforce Platform (https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/).

Upsert Syntax

```
upsert sObject | sObject[]

upsert sObject | sObject[] field
```

The optional field is a field token. For example, to specify the `MyExternalID` copy field, the statement is:

existing one:

- If the key is not matched, a new object record is created.
- If the key is matched once, the existing object record is updated.
- If the key is matched multiple times, an error is generated and the object record is neither inserted or updated.

Apex Basics & Database / content/learn/modules/apex_database?
trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)
Manipulate Records with DML

This example shows how upsert updates an existing contact record and inserts a new contact in one call. This upsert call updates the existing Josh contact and inserts a new contact, Kathy.

The upsert call uses the ID to match the first contact. The `josh` copy variable is being reused for the upsert call. This variable has already been populated with the record ID from the previous insert call, so the ID doesn't need to be set explicitly in this example.

1. Execute this snippet in the Execute Anonymous window of the Developer Console.

```
// Insert the Josh contact
Contact josh = new
Contact(FirstName='Josh',LastName='Kaplan',Department='Finan
ce');
insert josh;
// Josh's record has been inserted
//   so the variable josh has now an ID
//   which will be used to match the records by upsert
josh.Description = 'Josh\'s record has been updated by the
upsert operation.';
// Create the Kathy contact, but don't persist it in the
database
Contact kathy = new
Contact(FirstName='Kathy',LastName='Brown',Department='Techn
ology');
// List to hold the new contacts to upsert
List<Contact> contacts = new List<Contact> { josh, kathy };
// Call upsert
upsert contacts;
// Result: Josh is updated and Kathy is created.
```

Alternatively, you can specify a field to be used for matching records. This example uses the Email field on Contact because it has the `idLookup` `copy` property set. The example inserts the latest Jane Smith contact, and creates a second Contact sObject, populates it with the same email, then calls `upsert` `copy` to update the contact by using the email field for matching.

Manipulate Records with DML

If `insert` `copy` was used in this example instead of `upsert` `copy`, a duplicate Jane Smith contact would have been inserted.

1. Execute this snippet in the Execute Anonymous window of the Developer Console.

```
Contact jane = new Contact(FirstName='Jane',
                           LastName='Smith',
                           Email='jane.smith@example.com',
                           Description='Contact of the day');
insert jane;
// 1. Upsert using an idLookup field
// Create a second sObject variable.
// This variable doesn't have any ID set.
Contact jane2 = new Contact(FirstName='Jane',
                           LastName='Smith',
                           Email='jane.smith@example.com',
                           Description='Prefers to be
contacted by email.');
// Upsert the contact by using the idLookup field for
matching.
upsert jane2 Contact.fields.Email;
// Verify that the contact has been updated
System.assertEquals('Prefers to be contacted by email.',
                    [SELECT Description FROM Contact WHERE
Id=:jane.Id].Description);
```

2. Inspect all contacts in your org.
   Your org will have only one Jane Smith contact with the updated description.

id=xcloud.recycle_bin.htm&type=5) for 15 days, from where they can be restored. After 15 days, the records are scheduled for permanent deletion. Salesforce doesn't guarantee the exact time that the Recycle Bin permanently deletes these items.

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)
Manipulate Records with DML

This example shows how to delete all contacts whose last name is Smith. If you've run the sample for bulk DML, your org should already have two contacts with the last name of Smith. Execute this snippet in the Developer Console using Anonymous Apex, and then verify that there are no contacts with the last name Smith anymore.

```
Contact[] contactsDel = [SELECT Id FROM Contact WHERE
LastName='Smith'];
delete contactsDel;
```

This snippet includes a query to retrieve the contacts (a SOQL query). You'll learn more about SOQL in another unit.

DML Statement Exceptions

If a DML operation fails, it returns an exception of type `DmlException` copy. You can catch exceptions in your code to handle error conditions.

This example produces a `DmlException` copy because it attempts to insert an account without the required Name field. The exception is caught in the catch block.

```
try {
    // This causes an exception because
    //   the required Name field is not provided.
    Account acct = new Account();
    // Insert the account
    insert acct;
} catch (DmlException e) {
    System.debug('A DML exception has occurred: ' +
                e.getMessage());
}
```

These Database methods are static and are called on the class name.

- `Database.insert()` copy
- `Database.update()` copy
- `Database.upsert()` copy
- `Database.delete()` copy
- `Database.undelete()` copy
- `Database.merge()` copy (available only for the Lead, Contact, Case, and Account sObjects)

Unlike DML statements, Database methods have an optional `allOrNone` copy parameter that allows you to specify whether the operation should partially succeed. When this parameter is set to `false` copy, if errors occur on a partial set of records, the successful records will be committed and errors will be returned for the failed records. Also, no exceptions are thrown with the partial success option.

This is how you call the `insert` copy method with the `allOrNone` copy set to `false` copy.

```
Database.insert(recordList, false);
```

The Database methods return result objects containing success or failure information for each record. For example, insert and update operations each return an array of `Database.SaveResult` copy objects.

```
Database.SaveResult[] results = Database.insert(recordList,
false);
```

Upsert returns `Database.UpsertResult` copy objects, and delete returns `Database.DeleteResult` copy objects.

By default, the `allOrNone` copy parameter is `true` copy, which means that the Database method behaves like its DML statement counterpart and will throw an exception if a failure is encountered.

The following two statements are equivalent to the `insert recordList;` copy statement.

```
Database.insert(recordList, true);
```

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Beyond the Basics

Manipulate Records with DML

In addition to these methods, the Database class contains methods that aren't provided as DML statements. For example, methods used for transaction control and rollback, for emptying the Recycle Bin, and methods related to SOQL queries. You'll learn about SOQL in another unit.

Example: Insert Records with Partial Success

Let's take a look at an example that uses the Database methods. This example is based on the bulk DML example, but replaces the DML statement with a Database method. The `Database.insert()` copy method is called with the partial success option. One contact in the list doesn't have any fields on purpose and will cause an error because the contact can't be saved without the required LastName field. Three contacts are committed and the contact without any fields generates an error. The last part of this example iterates through the returned results and writes debug messages to the debug log.

1. Execute this example in the Execute Anonymous window of the Developer Console.

```
// Create a list of contacts
List<Contact> conList = new List<Contact> {
        new
Contact(FirstName='Joe',LastName='Smith',Department='Finance
'),
        new
Contact(FirstName='Kathy',LastName='Smith',Department='Techn
ology'),
        new
Contact(FirstName='Caroline',LastName='Roth',Department='Fin
ance'),
        new Contact()};
// Bulk insert all contacts with one DML call
Database.SaveResult[] srList = Database.insert(conList,
false);
// Iterate through each returned result
for (Database.SaveResult sr : srList) {

```

Show More

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

2. Verify the debug messages (use the DEBUG keyword for the filter).
   One failure should be reported and three contacts should have been inserted.

Manipulate Records with DML

Should You Use DML Statements or Database Methods?

- Use DML statements if you want any error that occurs during bulk DML processing to be thrown as an Apex exception that immediately interrupts control flow (by using `try. . .catch` `copy` blocks). This behavior is similar to the way exceptions are handled in most database procedural languages.
- Use Database class methods if you want to allow partial success of a bulk DML operation–if a record fails, the remainder of the DML operation can still succeed. Your application can then inspect the rejected records and possibly retry the operation. When using this form, you can write code that never throws DML exception errors. Instead, your code can use the appropriate results array to judge success or failure. Note that Database methods also include a syntax that supports thrown exceptions, similar to DML statements.

Work with Related Records

Create and manage records that are related to each other through relationships.

Insert Related Records

You can insert records related to existing records if a relationship has already been defined between the two objects, such as a lookup or master-detail relationship. A record is associated with a related record through a foreign key ID. For example, if inserting a new contact, you can specify the contact's related account record by setting the value of the `AccountId` `copy` field.

This example shows how to add a contact to an account (the related record) by setting the `AccountId` `copy` field on the contact. Contact and Account are linked through a lookup relationship.

1. Execute this snippet in the Anonymous Apex window of the Developer Console.

```
Account acct = new Account(Name='SFDC Account');
insert acct;
// Once the account is inserted, the sObject will be
```

```
        FirstName='Mario',
        LastName='Ruiz',
        Phone='415.555.1212',
        AccountId=a01D000000IRt53IAD);
    insert mario;
```

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Manipulate Records with DML

2. Inspect the accounts in your org.
   A new account (SFDC Account) has been created and has the Mario Ruiz contact in the account's Contacts related list.

Update Related Records

Fields on related records can't be updated with the same call to the DML operation and require a separate DML call. For example, if inserting a new contact, you can specify the contact's related account record by setting the value of the `AccountId` `copy` field. However, you can't change the account's name without updating the account itself with a separate DML call. Similarly, when updating a contact, if you also want to update the contact's related account, you must make two DML calls. The following example updates a contact and its related account using two `update` `copy` statements.

```
// Query for the contact, which has been associated with an
account.
Contact queriedContact = [SELECT Account.Name
                          FROM Contact
                          WHERE FirstName = 'Mario' AND
LastName='Ruiz'
                          LIMIT 1];
// Update the contact's phone number
queriedContact.Phone = '(415)555-1213';
// Update the related account industry
queriedContact.Account.Industry = 'Technology';
// Make two separate calls
// 1. This call is to update the contact's phone.
update queriedContact;
// 2. This call is to update the related account's Industry
field.
update queriedContact.Account;
```

For example, deleting the account you created earlier (SFDC Account) will delete its related contact too.

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Manipulate Records with DML

1. Execute this snippet in the Anonymous Apex window of the Developer Console.

```
Account[] queriedAccounts = [SELECT Id FROM Account WHERE
Name='SFDC Account'];
delete queriedAccounts;
```

2. Check the accounts and contacts in your org.
   You'll see that both the account and its related contact were deleted.

About Transactions

DML operations execute within a transaction. All DML operations in a transaction either complete successfully, or if an error occurs in one operation, the entire transaction is rolled back and no data is committed to the database. The boundary of a transaction can be a trigger, a class method, an anonymous block of code, an Apex page, or a custom Web service method. For example, if a trigger or class creates two accounts and updates one contact, and the contact update fails because of a validation rule failure, the entire transaction rolls back and none of the accounts are persisted in Salesforce.

Resources

- *Apex Developer Guide*: Working with Data in Apex (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_data_intro.htm)
- *Apex Developer Guide*: Apex DML Operations (https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_dml_section.htm)
- *Apex Reference Guide*: Database Class (https://developer.salesforce.com/docs/atlas.en-us.apexref.meta/apexref/apex_methods_system_database.htm)
- *Apex Developer Guide*: Exception Statements (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_exception_statements.htm)
- *Apex Developer Guide*: sObjects That Don't Support DML Operations (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_dml_non_dml_objects.htm)

YOUR CHALLENGE

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Manipulate Records with DML

Create a method for inserting accounts

For this challenge, create an Apex class that inserts a new account named after an incoming parameter. If the account is successfully inserted, the method should return the account record. If a DML exception occurs, the method should return null.

- The Apex class must be called **AccountHandler** and be in the public scope
- The Apex class must have a public static method called **insertNewAccount**
  - The method must accept an incoming string as a parameter, which will be used to create the Account name
  - The method must insert the account into the system and then return the record
  - The method must also accept an empty string, catch the failed DML and then return null

Choose hands-on org

Koushik nelluri Salesforce CRM
Created on 5/9/2026

Launch

Check Challenge to Earn 500 Points

Report Illegal Content (EU)

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Download Trailhead GO

Manipulate Records with DML